



Dynamic Programming #2

Presenter: Dr. Ha Viet Uyen Synh.



BINOMIAL COEFFICIENT



Binomial Coefficient

DEFINITION

Binomial coefficients are the ones that appear as the coefficient of powers of x in the expansion of $(1 + x)^n$:

$$(1 + x)^n = n_{c_0} + n_{c_1}x + n_{c_2}x^2 + \cdots + n_{c_n}x^n,$$

where $n_{c_k} = \frac{n!}{k!(n-k)!}$. The terms from n_{c_0} to n_{c_n} are called as the binomial coefficients. Hereafter the term n_{c_k} will be written as $\binom{n}{k}$.

EXAMPLE

Calculate the value of $\binom{7}{3}$.

We can use the formula $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ to calculate the value of $\binom{7}{3}$:

$$\binom{7}{3} = \frac{7!}{3! \times (7-3)!} = \frac{5040}{6 \times 24} = 35. \square$$

Binomial coefficients are equal to the number of ways there are to choose k objects from a set of n objects.

Exercises

1. How many ways can you select 2 balls from a box containing 9 balls?
2. How many 3-digit numbers can be formed by using 1, 2, 3, 4, 5 without repetition of digits?
3. How many ways could you choose a three-topping pizza based on the following menu?

Build your own pizza!

Pizza toppings

Pepperoni

Sausage

Ham

Bacon

Chicken

Onion

Mushroom

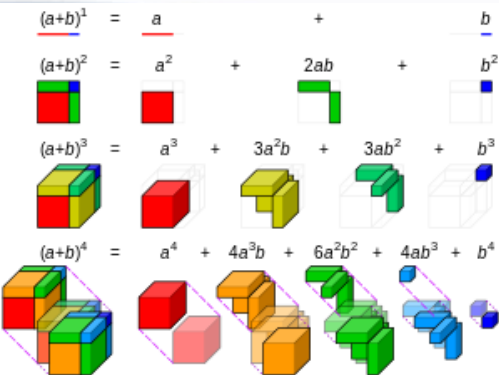
Pepper

Olives

Pineapple



Binomial Expansion Using Coefficients



DEFINITION

When given a binomial, $(x + y)^a$, you may expand the binomial using the following equation:

$$(x + y)^a = \binom{a}{0} x^a y^0 + \binom{a}{1} x^{(a-1)} y^1 + \cdots + \binom{a}{a-1} x^1 y^{(a-1)} + \binom{a}{a} x^0 y^a.$$

This is a simplified definition of **binomial expansion** using binomial coefficients.

EXAMPLE

Find the coefficient of x^7 in the expansion of $(x + 3)^{10}$.

Recall that the binomial theorem is

$$(x + y)^a = \binom{a}{0} x^a y^0 + \binom{a}{1} x^{(a-1)} y^1 + \cdots + \binom{a}{a-1} x^1 y^{(a-1)} + \binom{a}{a} x^0 y^a,$$

which in summation form is

$$(x + y)^a = \sum_{i=0}^a \binom{a}{i} x^{a-i} y^i.$$

In order to get the coefficient of x^7 , we need to have $a - i = 7$. Since $a = 10$, $i = 3$.

Thus, the answer is $y^3 \binom{a}{3} = 3^3 \binom{10}{3} = 3240$. $\square$



Exercises

4. Find the number of digits of the coefficient of x^{56} in the product below:

$$A = \prod_{n=0}^{59} (x + 2n + 1)$$

5. Three numbers, $x, y, z \in \mathbb{N}$ are chosen at random such that $1 \leq x, y, z \leq 10$. What is the probability that their product is divisible by 9?



Problem

Write a function that takes two parameters n and k and returns the value of Binomial Coefficient $C(n, k)$.

For example, your function should return 6 for $n = 4$ and $k = 2$, and it should return 10 for $n = 5$ and $k = 2$.

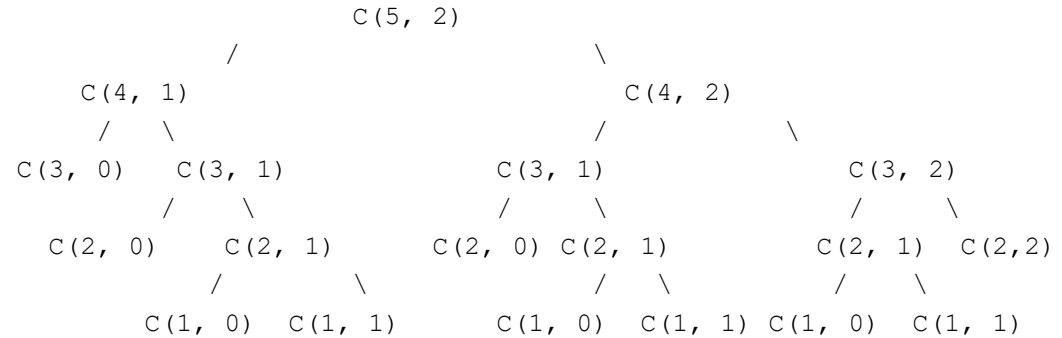


1) Optimal Substructure

$$C(n, k) = C(n-1, k-1) + C(n-1, k)$$

$$C(n, 0) = C(n, n) = 1$$

2) Overlapping Subproblems





Implementation

```
// A naive recursive C++ implementation
#include <bits/stdc++.h>
using namespace std;

// Returns value of Binomial Coefficient C(n, k)
int binomialCoeff(int n, int k) {
    if (k == 0 || k == n)
        return 1;
    return binomialCoeff(n - 1, k - 1) + binomialCoeff(n - 1, k);
}

/* Driver code */
int main()
{
    int n = 5, k = 2;
    cout << "Value of C("<n<< ", "<k<< ") is " << binomialCoeff(n, k);
    return 0;
}
```

A stack of five smooth, dark blue stones is balanced on a reflective surface, creating a serene and minimalist aesthetic. The stones are stacked horizontally, with the top stone being the smallest and the bottom one the largest. The surface they rest on is highly reflective, mirroring the stones and the light from the background. The background is a soft, out-of-focus blue and white, suggesting a calm, possibly aquatic environment.

```
#include<bits/stdc++.h>
using namespace std;
int min(int a, int b) { return (a < b) ? a : b; }
int binomialCoeff(int n, int k) {
    int C[n + 1][k + 1];
    int i, j;
    for (i = 0; i <= n; i++) {
        for (j = 0; j <= min(i, k); j++) {
            if (j == 0 || j == i)    C[i][j] = 1;
            else    C[i][j] = C[i - 1][j - 1] + C[i - 1][j];
        }
    }
    return C[n][k];
}

int main() {
    int n = 5, k = 2;
    cout << "Value of C[" << n << "]["
        << k << "] is " << binomialCoeff(n, k);
}
```



// Solution of Binomial Coefficient

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int binomialCoeff(int n, int k) {
```

```
    int C[k+1];
```

```
    memset(C, 0, sizeof(C));
```

```
    C[0] = 1; // nC0 is 1
```

```
    for (int i = 1; i <= n; i++)      {
```

```
        // Compute next row of pascal triangle using the previous row
```

```
        for (int j = min(i, k); j > 0; j--)
```

```
            C[j] = C[j] + C[j-1];
```

```
    }
```

```
    return C[k];
```

```
}
```

```
int main() {
```

```
    int n = 5, k = 2;
```

```
    printf ("Value of C(%d, %d) is %d ", n, k, binomialCoeff(n, k) );
```

```
    return 0;
```

```
}
```



LONGEST COMMON SUBSEQUENCE



Task

The longest common subsequence (LCS) is defined as the longest subsequence that is common to all the given sequences, provided that the elements of the subsequence are not required to occupy consecutive positions within the original sequences.

If $S1$ and $S2$ are the two given sequences then, Z is the common subsequence of $S1$ and $S2$ if Z is a subsequence of both $S1$ and $S2$. Furthermore, Z must be a strictly increasing sequence of the indices of both $S1$ and $S2$.

Ex:

$S1 = \{B, C, D, A, A, C, D\}$

$S2 = \{A, C, D, B, A, C\}$

Common subsequences are $\{B, C\}$, $\{C, D, A, C\}$, $\{D, A, C\}$, $\{A, A, C\}$, $\{A, C\}$, $\{C, D\}$,...

Among these subsequences, $\{C, D, A, C\}$ is the longest common subsequence.

Using Dynamic Programming to find the LCS

Let us take two sequences:

X

A

C

A

D

B

The first sequence

Y

C

B

D

A

Second Sequence



1. Create a table of dimension $(n+1)*(m+1)$ where n and m are the lengths of X and Y respectively. The first row and the first column are filled with zeros.

		C	B	D	A
	0	0	0	0	0
A	0				
C	0				
A	0				
D	0				
B	0				

Initialise a table



Fill each cell of the table using the following logic:

- If the character corresponding to the *current row* and *current column* are matching, then fill the current cell by adding one to the diagonal element. Point an arrow to the diagonal cell.
- Else take the maximum value from the *previous column* and *previous row* element for filling the current cell. Point an arrow to the cell with maximum value. If they are equal, point to any of them.

		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0				
A	0				
D	0				
B	0				

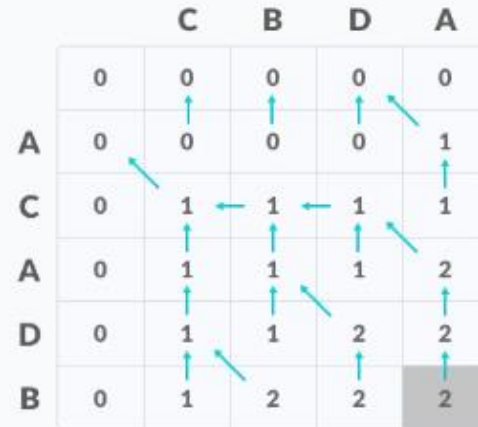
Fill the values

Step 2 is repeated until the table is filled.

		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0	1	1	1	1
A	0	1	1	1	2
D	0	1	1	2	2
B	0	1	2	2	2

Fill all the values

The value in the last row and the last column is the length of the longest common subsequence.

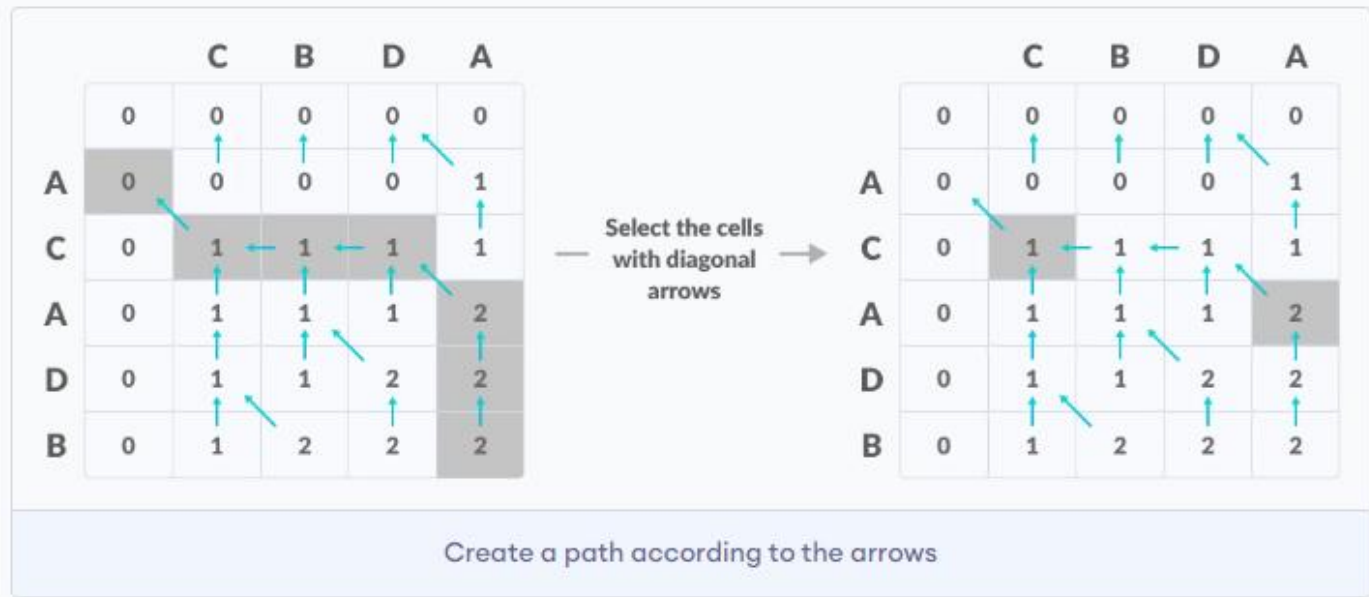


		C	B	D	A
	0	0	0	0	0
A	0	0	0	0	1
C	0	1	1	1	1
A	0	1	1	1	2
D	0	1	1	2	2
B	0	1	2	2	2

The bottom right corner is the length of the LCS



In order to find the longest common subsequence, start from the last element and follow the direction of the arrow. The elements corresponding to () symbol form the longest common subsequence.



Thus, the longest common subsequence is CA.



Longest Common Subsequence Algorithm

X and Y be two given sequences

Initialize a table LCS of dimension $X.length * Y.length$

X.label = X

Y.label = Y

LCS[0][] = 0

LCS[][0] = 0

Start from LCS[1][1]

Compare X[i] and Y[j]

 If $X[i] = Y[j]$

$LCS[i][j] = 1 + LCS[i-1, j-1]$

 Point an arrow to LCS[i][j]

 Else

$LCS[i][j] = \max(LCS[i-1][j], LCS[i][j-1])$

 Point an arrow to $\max(LCS[i-1][j], LCS[i][j-1])$



Implementation

```
#include <iostream>
using namespace std;
void lcsAlgo(char *S1, char *S2, int m, int n)
{
    int LCS_table[m + 1][n + 1];
    // Building the matrix in bottom-up way
    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0)
                LCS_table[i][j] = 0;
            else if (S1[i - 1] == S2[j - 1])
                LCS_table[i][j] = LCS_table[i - 1][j - 1] +
2;
            else
                LCS_table[i][j] = max(LCS_table[i - 1][j],
LCS_table[i][j - 1]);
        }
    }
}
```

```
int index = LCS_table[m][n];
char lcsAlgo[index + 1];
lcsAlgo[index] = '\0';
int i = m, j = n;
while (i > 0 && j > 0) {
    if (S1[i - 1] == S2[j - 1]) {
        lcsAlgo[index - 1] = S1[i - 1];
        i--; j--; index--;
    }
    else if (LCS_table[i - 1][j] > LCS_table[i][j - 1])
        i--;
    else
        j--;
}
// Printing the sub sequences
cout << "S1 : " << S1 << "\nS2 : " << S2 <<
"\nLCS: " << lcsAlgo << "\n";
}
```



```
int main() {  
    char S1[] = "ACADB";  
    char S2[] = "CBDA";  
    int m = strlen(S1);  
    int n = strlen(S2);  
  
    lcsAlgo(S1, S2, m, n);  
}
```



GOLD MINE PROBLEM

Task

Given a gold mine of $n \times m$ dimensions. Each field in this mine contains a positive integer which is the amount of gold in tons. Initially the miner is at first column but can be at any row. He can move only (right->,right up /,right down\) that is from a given cell, the miner can move to the cell diagonally up towards the right or right or diagonally down towards the right. Find out maximum amount of gold he can collect.

1	5	12
2	4	4
0	6	4
3	0	0

Input : `mat[][] = {{1, 3, 3},
 {2, 1, 4},
 {0, 6, 4}};`

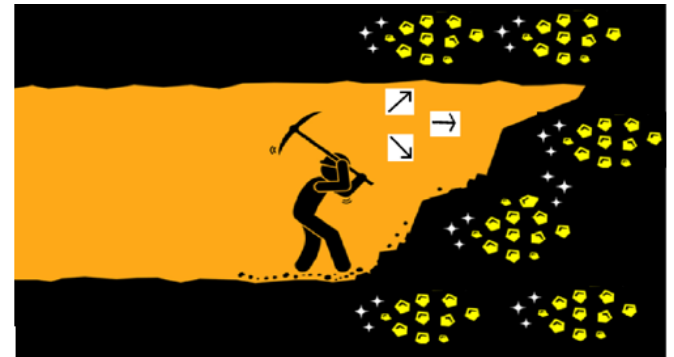
Output : 12
`{{(1,0)->(2,1)->(2,2)}`

Input: `mat[][] = { {1, 3, 1, 5},
 {2, 2, 4, 1},
 {5, 0, 2, 3},
 {0, 6, 1, 2}};`

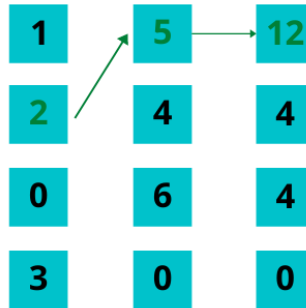
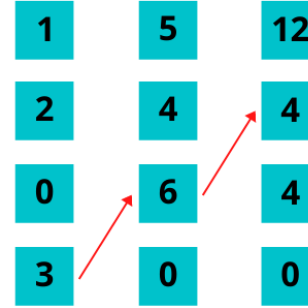
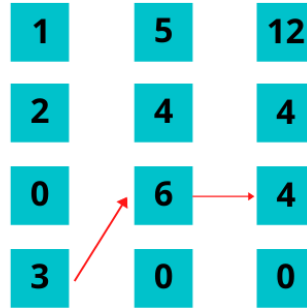
Output : 16
`(2,0) -> (1,1) -> (1,2) -> (0,3) OR`
`(2,0) -> (3,1) -> (2,2) -> (2,3)`

Input : `mat[][] = {{10, 33, 13, 15},
 {22, 21, 04, 1},
 {5, 0, 2, 3},
 {0, 6, 14, 2}};`

Output : 83



Greedy approach:



```
1  Algorithm Greedy( $a, n$ )
2  //  $a[1 : n]$  contains the  $n$  inputs.
3  {
4       $solution := \emptyset$ ; // Initialize the solution.
5      for  $i := 1$  to  $n$  do
6          {
7               $x := \text{Select}(a)$ ;
8              if Feasible( $solution, x$ ) then
9                   $solution := \text{Union}(solution, x)$ ;
10         }
11     return  $solution$ ;
12 }
```

Recursive formula

$$\text{Max_Gold}[i][j] = \text{Gold_Mine}[i][j] + \max \{ \begin{array}{l} \text{Max_Gold}[i], [j-1] , \\ \text{Max_Gold}[i-1], [j-1] , \\ \text{Max_Gold}[i+1], [j-1] \} \end{array}$$

for all possible values

1	0	0
2	0	0
0	0	0
3	0	0

1	7	0	1	7	19	1	7	19
2	6	0	2	6	13	2	6	13
0	9	0	0	9	13	0	9	13
3	3	0	3	3	9	3	3	9



```
#include <bits/stdc++.h>
using namespace std;

int GoldMine(int** arr, int n, int m){
    int DP[n][m];
    memset(DP, 0, sizeof(DP));
    for (int i = 0; i < n; i++)
        DP[i][0] = arr[i][0];
    //for every column
    for (int j = 1; j < m; j++) {
        //check which option is better accordingly
        for (int i = 0; i < n; i++) {
            DP[i][j] = arr[i][j];
            int val = DP[i][j - 1];
            if (i - 1 >= 0) {
                if (val < DP[i - 1][j - 1])
                    val = DP[i - 1][j - 1];
            }
            if (i + 1 < n) {
                if (val < DP[i + 1][j - 1])
                    val = DP[i + 1][j - 1];
            }
            DP[i][j] += val;
        }
    }
    int gold = DP[0][m - 1];
    for (int i = 1; i < n; i++) {
        if (DP[i][m - 1] > gold)
            gold = DP[i][m - 1];
    }
    return gold;
}
```

```
int main()
{
    int n, item, m;

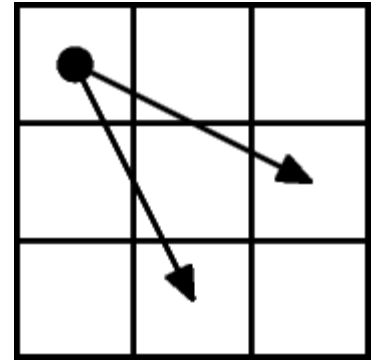
    cout << "Enter matrix dimensions, m & n\n";
    cin >> n >> m;

    cout << "Input matrix cells\n";
    int** arr = (int**)(malloc(sizeof(int*) * n));

    //input array
    for (int j = 0; j < n; j++) {
        arr[j] = (int*)(malloc(sizeof(int) * m));
        for (int k = 0; k < m; k++)
            cin >> arr[j][k];
    }

    cout << "Max amount of gold that can be
    collected: " << GoldMine(arr, n, m) << endl;

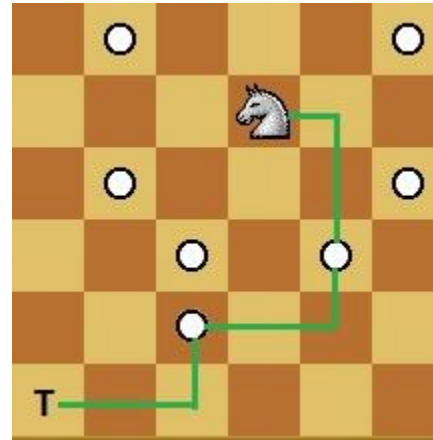
    return 0;
}
```



A KNIGHT

Minimum steps to reach target by a Knight

Given a square chessboard of $N \times N$ size, the position of Knight and position of a target is given. We need to find out the minimum steps a Knight will take to reach the target position.



In above diagram Knight takes 3 step to reach from (4, 5) to (1, 1) (4, 5) $\rightarrow$ (5, 3) $\rightarrow$ (3, 2) $\rightarrow$ (1, 1) as shown in diagram



TILING PROBLEM

1. Tiling Problem

Given a “ $2 \times n$ ” board and tiles of size “ 2×1 ”, count the number of ways to tile the given board using the 2×1 tiles. A tile can either be placed horizontally i.e., as a 1×2 tile or vertically i.e., as 2×1 tile.

Ex:

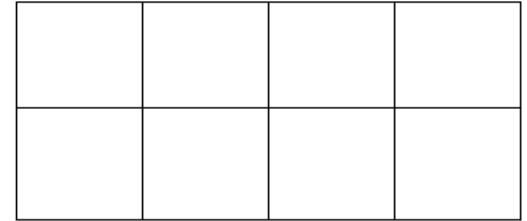
Input $n = 4$

Output: 5

Explanation:

For a 2×4 board, there are 5 ways

- 1) All 4 vertical
- 2) All 4 horizontal
- 3) First 2 vertical, remaining 2 horizontal
- 4) First 2 horizontal, remaining 2 vertical
- 5) Corner 2 vertical, middle 2 horizontal



Board



Tile



Implementation

Let “count(n)” be the count of ways to place tiles on a “2 x n” grid, we have following two ways to place first tile.

- 1) If we place first tile vertically, the problem reduces to “count(n-1)”
- 2) If we place first tile horizontally, we have to place second tile also horizontally. So the problem reduces to “count(n-2)”

Therefore, count(n) can be written as below.

$$\text{count}(n) = n \text{ if } n = 1 \text{ or } n = 2$$

$$\text{count}(n) = \text{count}(n-1) + \text{count}(n-2)$$



2. Count the number of ways to tile the floor of size $n \times m$ using $1 \times m$ size tiles

Given a floor of size $n \times m$ and tiles of size $1 \times m$. The problem is to count the number of ways to tile the given floor using $1 \times m$ tiles. A tile can either be placed horizontally or vertically. Both n and m are positive integers and $2 \leq m$.

Examples:

Input : $n = 2, m = 3$

Output : 1

Only one combination to place
two tiles of size 1×3 horizontally
on the floor of size 2×3 .

Input : $n = 4, m = 4$

Output : 2

1st combination:

All tiles are placed horizontally

2nd combination:

All tiles are placed vertically.



For a given value of n and m , the number of ways to tile the floor can be obtained from the following relation.

$$\text{count}(n) = \begin{cases} 1, & 1 \leq n < m \\ 2, & n = m \\ \text{count}(n-1) + \text{count}(n-m), & m < n \end{cases}$$



```
#include <bits/stdc++.h>
using namespace std;
// function to count the total number of ways
int countWays(int n, int m) {

    // table to store values of subproblems
    int count[n + 1];
    count[0] = 0;

    // Fill the table upto value n
    for (int i = 1; i <= n; i++) {
        // recurrence relation
        if (i > m)
            count[i] = count[i - 1] + count[i - m];
        // base cases and for i = m = 1
        else if (i < m || i == 1)
            count[i] = 1;
        // i == m
        else
            count[i] = 2;
    }
    // required number of ways
    return count[n];
}

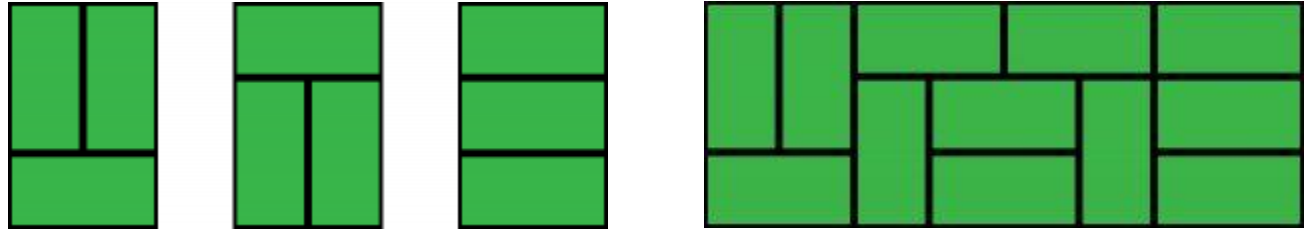
// Driver program to test above
int main()
{
    int n = 7, m = 4;
    cout << "Number of ways = " << countWays(n, m);
    return 0;
}
```

3. Tiling with Dominoes

Given a $3 \times n$ board, find the number of ways to fill it with 2×1 dominoes.

Example 1

Following are all the 3 possible ways to fill up a 3×2 board.



Example 2

Here is one possible way of filling a 3×8 board. You have to find all the possible ways to do so.

Output: 153

Defining Subproblems

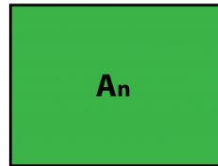
At any point while filling the board, there are three possible states that the last column can be in:

A_n = No. of ways to completely fill a $3 \times n$ board. (We need to find this)

B_n = No. of ways to fill a $3 \times n$ board with top corner in last column not filled.

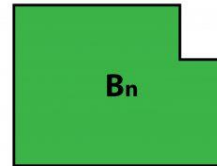
C_n = No. of ways to fill a $3 \times n$ board with bottom corner in last column not filled.

Possible states of the last column



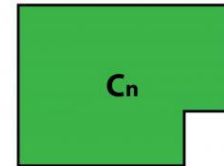
A_n

Completely Filled



B_n

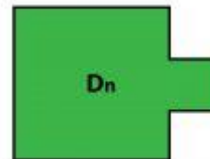
Top Corner Empty



C_n

Bottom Corner Empty

Following States are NOT possible



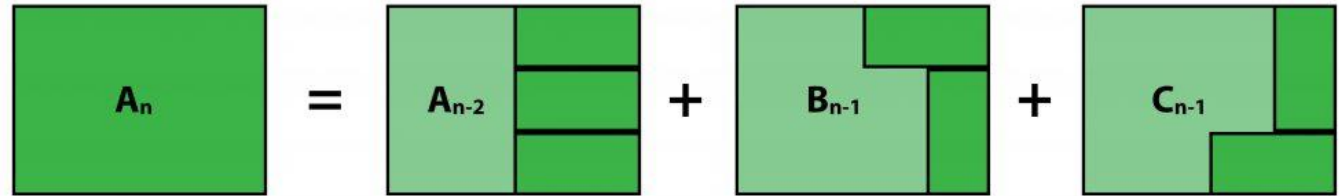
D_n



E_n

Recurrences

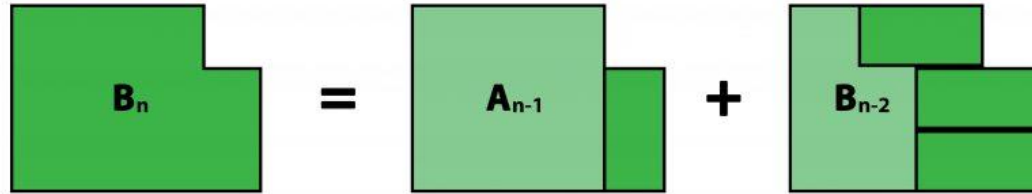
Calculating A_n :



$$A_n = A_{n-2} + B_{n-1} + C_{n-1}$$

$$A_n = A_{n-2} + 2 * B_{n-1}$$

Calculating B_n :



$$B_n = A_{n-1} + B_{n-2}$$

Final Recursive Relations are:

$$A_n = A_{n-2} + 2*B_{n-1}$$

$$B_n = A_{n-1} + B_{n-2}$$

Base cases:

$$A_0 = 1 ; B_0 = 0$$

$$A_1 = 0 ; B_1 = 1$$



```
#include <iostream>
using namespace std;

int countWays(int n)
{
    int A[n + 1], B[n + 1];
    A[0] = 1, A[1] = 0, B[0] = 0, B[1] = 1;
    for (int i = 2; i <= n; i++) {
        A[i] = A[i - 2] + 2 * B[i - 1];
        B[i] = A[i - 1] + B[i - 2];
    }

    return A[n];
}

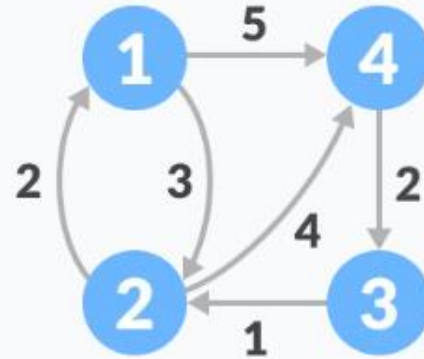
int main()
{
    int n = 8;
    cout << countWays(n);
    return 0;
}
```



FLOYD-WARSHALL ALGORITHM

Task

Floyd-Warshall Algorithm is an algorithm for finding the shortest path between all the pairs of vertices in a weighted graph. This algorithm works for both the directed and undirected weighted graphs. But, it does not work for the graphs with negative cycles (where the sum of the edges in a cycle is negative).



Initial graph

How Floyd-Warshall Algorithm Works?

1. Create a matrix A^0 of dimension $n \times n$ where n is the number of vertices. The row and the column are indexed as i and j respectively. i and j are the vertices of the graph.

Each cell $A[i][j]$ is filled with the distance from the i^{th} vertex to the j^{th} vertex. If there is no path from i^{th} vertex to j^{th} vertex, the cell is left as infinity.

$$A^0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & \infty & 5 \\ 2 & 0 & \infty & 4 \\ \infty & 1 & 0 & \infty \\ \infty & \infty & 2 & 0 \end{bmatrix} \end{matrix}$$

Fill each cell with the distance between i^{th} and j^{th} vertex



2. Now, create a matrix A^1 using matrix A^0 . The elements in the first column and the first row are left as they are. The remaining cells are filled in the following way:

Let k be the intermediate vertex in the shortest path from source to destination. In this step, k is the first vertex. $A[i][j]$ is filled with $(A[i][k] + A[k][j])$ if $(A[i][j] > A[i][k] + A[k][j])$.

That is, if the direct distance from the source to the destination is greater than the path through the vertex k , then the cell is filled with $A[i][k] + A[k][j]$.

In this step, k is vertex 1. We calculate the distance from source vertex to destination vertex through this vertex k .

For example: For $A^1[2, 4]$, the direct distance from vertex 2 to 4 is 4 and the sum of the distance from vertex 2 to 4 through vertex (ie. from vertex 2 to 1 and from vertex 1 to 4) is 7. Since $4 < 7$, $A^0[2, 4]$ is filled with 4.



Floyd-Warshall Algorithm

n = no of vertices

A = matrix of dimension $n \times n$

for $k = 1$ to n

 for $i = 1$ to n

 for $j = 1$ to n

$A^k[i, j] = \min (A^{k-1}[i, j], A^{k-1}[i, k] + A^{k-1}[k, j])$

return A



Implementation

```
#include <iostream>
using namespace std;
#define nV 4
#define INF 999
void printMatrix(int matrix[][nV]);
void floydWarshall(int graph[][nV]) {
    int matrix[nV][nV], i, j, k;
    for (i = 0; i < nV; i++)
        for (j = 0; j < nV; j++)
            matrix[i][j] = graph[i][j];
    // Adding vertices individually
    for (k = 0; k < nV; k++) {
        for (i = 0; i < nV; i++) {
            for (j = 0; j < nV; j++) {
                if (matrix[i][k] + matrix[k][j] < matrix[i][j])
                    matrix[i][j] = matrix[i][k] + matrix[k][j];
            }
        }
    }
    printMatrix(matrix);
}
```

```
void printMatrix(int matrix[][nV]) {
    for (int i = 0; i < nV; i++) {
        for (int j = 0; j < nV; j++) {
            if (matrix[i][j] == INF)
                printf("%4s", "INF");
            else
                printf("%4d", matrix[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int graph[nV][nV] = {{0, 3, INF, 5},
                        {2, 0, INF, 4},
                        {INF, 1, 0, INF},
                        {INF, INF, 2, 0}};
    floydWarshall(graph);
}
```

